

TASK 02 — DTOs, Request Validation & API Error Standards

Step 1 — Create Request DTO

This class represents the **data required to create a new product**.

```
package com.example.productservice.dto;

import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import java.math.BigDecimal;

public class CreateProductRequest {

    @NotBlank(message = "Product name is required")
    private String name;

    private String description;

    @NotNull(message = "Price is required")
    @DecimalMin(value = "0.01", message = "Price must be greater than 0")
    private BigDecimal price;

    @NotBlank(message = "Category is required")
    private String category;

    @NotNull(message = "Stock is required")
    @Min(value = 0, message = "Stock cannot be negative")
    private Integer stock;

    // No-argument constructor
    public CreateProductRequest() {
    }

    // Getters and Setters
    public String getName() {
```

```
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public BigDecimal getPrice() {
        return price;
    }

    public void setPrice(BigDecimal price) {
        this.price = price;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    public Integer getStock() {
        return stock;
    }
}
```

```
public void setStock(Integer stock) {  
    this.stock = stock;  
}  
}
```

Example:

```
{  
    "name": "",  
    "price": -100,  
    "stock": -5  
}
```

The API should reject this request.

Step 2 — Create Update DTO

This class represents the **data required to update an existing product**.

For example: PUT /api/products/1

The client wants to change:

Laptop → Gaming Laptop

55000 → 65000

Instead of using the database entity directly, we use:

UpdateProductRequest

To create it

src/main/java/com/example/productservice/dto/UpdateProductRequest.java

Code

```
package com.example.productservice.dto;  
  
import jakarta.validation.constraints.DecimalMin;  
import jakarta.validation.constraints.Min;  
import jakarta.validation.constraints.NotBlank;
```

```
import jakarta.validation.constraints.NotNull;
import java.math.BigDecimal;
public class UpdateProductRequest {
    @NotBlank(message = "Product name is required")
    private String name;
    private String description;
    @NotNull(message = "Price is required")
    @DecimalMin(value = "0.01", message = "Price must be greater than 0")
    private BigDecimal price;
    @NotBlank(message = "Category is required")
    private String category;
    @NotNull(message = "Stock is required")
    @Min(value = 0, message = "Stock cannot be negative")
    private Integer stock;
    // No-argument constructor
    public UpdateProductRequest() {
    }
    // Getters and Setters
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getDescription() {
        return description;
    }
    public void setDescription(String description) {
```

```

        this.description = description;
    }
    public BigDecimal getPrice() {
        return price;
    }
    public void setPrice(BigDecimal price) {
        this.price = price;
    }
    public String getCategory() {
        return category;
    }
    public void setCategory(String category) {
        this.category = category;
    }
    public Integer getStock() {
        return stock;
    }
    public void setStock(Integer stock) {
        this.stock = stock;
    }
}

```

Create DTO

Used for creating a product

POST /api/products

Creates new data

CreateProductRequest

Update DTO

Used for updating a product

PUT /api/products/{id}

Changes existing data

UpdateProductRequest

Example

Create:

```
{  
  "name": "Laptop",  
  "price": 55000  
}
```

Update:

```
{  
  "name": "Gaming Laptop",  
  "price": 65000  
}
```

The fields are similar, but the DTOs represent **different operations**.

Step 3 — Create Response DTO

This class represents the **data that your API sends back to the client**.

For example, after creating a product:

POST /api/products

The API should return:

```
{  
  "id": 1,  
  "name": "Laptop",  
  "description": "Dell Laptop",  
  "price": 55000,  
  "category": "Electronics",  
  "stock": 10  
}
```

This is the **response DTO**.

Where to create it?

src/main/java/com/example/productservice/dto/ProductResponse.java

Code

```
package com.example.productservice.dto;

import java.math.BigDecimal;

public class ProductResponse {

    private Long id;

    private String name;

    private String description;

    private BigDecimal price;

    private String category;

    private Integer stock;

    // No-argument constructor
    public ProductResponse() {

    }

    // All-argument constructor
    public ProductResponse(Long id,
                           String name,
                           String description,
                           BigDecimal price,
                           String category,
                           Integer stock) {

        this.id = id;

        this.name = name;

        this.description = description;

        this.price = price;

        this.category = category;

        this.stock = stock;

    }

    // Getters and Setters
```

```
public Long getId() {  
    return id;  
}  
public void setId(Long id) {  
    this.id = id;  
}  
public String getName() {  
    return name;  
}  
public void setName(String name) {  
    this.name = name;  
}  
public String getDescription() {  
    return description;  
}  
public void setDescription(String description) {  
    this.description = description;  
}  
public BigDecimal getPrice() {  
    return price;  
}  
public void setPrice(BigDecimal price) {  
    this.price = price;  
}  
public String getCategory() {  
    return category;  
}  
public void setCategory(String category) {
```

```

        this.category = category;
    }
    public Integer getStock() {
        return stock;
    }
    public void setStock(Integer stock) {
        this.stock = stock;
    }
}

```

Step 4 — Stop exposing Entity directly

This task is asking you to **separate your database Entity from your REST API response**.

Do not send the database object directly to the client. Use a DTO to control the data.

Understand the Request Flow

Request



DTO



Service



Entity



Database

This means:

Client sends JSON → Controller receives DTO → Service processes it → Entity is saved in database.

The client sends:

```

{
    "name": "Laptop",
    "price": 50000,

```

```
"stock": 10
}
```

The controller receives this as a **ProductRequest DTO**.

Understand the Response Flow

Database



Entity



DTO



Response

Database returns Entity → Service converts Entity into DTO → Controller sends DTO to client.

The database contains:

id = 1

name = Laptop

price = 50000

stock = 10

But your API may return only:

```
{
  "id": 1,
  "name": "Laptop",
  "price": 50000
}
```

This is called **DTO mapping**.

we use DTO

Suppose your Entity contains:

```
Product {
    id;
```

```
    name;  
    price;  
    stock;  
    createdAt;  
}
```

If we return the Entity directly, all these fields may be exposed.

But with a DTO, you decide exactly what the client receives.

Entity

@Entity

```
public class Product {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    private String name;  
    private Double price;  
    private Integer stock;  
}
```

Response DTO

```
public class ProductResponse {  
    private Long id;  
    private String name;  
    private Double price;  
}
```

Entity = Database structure

DTO = API data structure

Step 5 — Add Validation

Validation means **checking whether the data sent by the user is correct before saving it to the database.**

we creating a product, we should not allow:

- An empty product name.
- A negative price.
- A negative stock.
- An empty category.

Spring Boot provides validation annotations to handle these checks.

Annotation	Meaning	Example
@NotBlank	String must not be null, empty, or only spaces.	Product name
@NotNull	Value must not be null.	Price or stock
@Positive	Number must be greater than 0.	Price
@PositiveOrZero	Number must be 0 or greater.	Stock
@Size	Checks the length of a string.	Name or category

Product DTO with validation

Create a file:

ProductRequest.java

```
package com.example.product.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.PositiveOrZero;
import jakarta.validation.constraints.Size;

public class ProductRequest {

    @NotBlank(message = "Product name is required")

    @Size(min = 2, max = 100,
        message = "Product name must be between 2 and 100 characters")
```

```
private String name;
@NotNull(message = "Price is required")
@Positive(message = "Price must be greater than 0")
private Double price;
@NotNull(message = "Stock is required")
@PositiveOrZero(message = "Stock cannot be negative")
private Integer stock;
@NotBlank(message = "Category is required")
@Size(min = 2, max = 50,
      message = "Category must be between 2 and 50 characters")
private String category;
public ProductRequest() {
}
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public Double getPrice() {
    return price;
}
public void setPrice(Double price) {
    this.price = price;
}
public Integer getStock() {
    return stock;
}
```

```
public void setStock(Integer stock) {  
    this.stock = stock;  
}  
public String getCategory() {  
    return category;  
}  
public void setCategory(String category) {  
    this.category = category;  
}  
}
```

Understand the validation line by line

Product name:

@NotBlank(message = "Product name is required")

This rejects:

"name": ""

It also rejects:

"name": " "

Because the name must contain actual text.

@Size(min = 2, max = 100)

This means the name must contain **at least 2 characters and at most 100 characters**.

Price:

@NotNull(message = "Price is required")

This rejects:

"price": null

@Positive(message = "Price must be greater than 0")

This rejects:

"price": -100

It also rejects:

"price": 0

Because the price must be **greater than 0**.

Stock:

@NotNull(message = "Stock is required")

Stock must be provided.

@PositiveOrZero(message = "Stock cannot be negative")

This allows:

"stock": 0

But rejects:

"stock": -5

A product can be out of stock, so stock 0 is valid.

Category:

@NotBlank(message = "Category is required")

This rejects an empty category.

@Size(min = 2, max = 50)

This ensures the category name has a valid length.

Step 6 — Test Invalid Input

Validation result

Field	Value	Result	Reason
name	""	Invalid	Name is blank
price	-100	Invalid	Price must be positive
stock	-5	Invalid	Stock cannot be negative
category	Missing	Invalid	Category is required

Expected response

The API should return:

400 Bad Request

Because the user sent invalid data.

Example response:

```
{  
  "status": 400,  
  "message": "Validation failed",  
  "errors": {  
    "name": "Product name is required",  
    "price": "Price must be greater than 0",  
    "stock": "Stock cannot be negative",  
    "category": "Category is required"  
  }  
}
```

The exact error response depends on how your project handles validation errors.

validation works in the application

User sends POST request



Controller receives request



@Valid checks ProductRequest



Are all fields valid?



YES

NO



Service saves

Return 400

product

Bad Request



Return 201 Created

Controller example

@PostMapping

```
public ResponseEntity<Product> createProduct(  
    @Valid @RequestBody ProductRequest request) {  
    Product product = productService.createProduct(request);  
    return ResponseEntity.status(HttpStatus.CREATED)  
        .body(product);  
}
```

@Valid @RequestBody ProductRequest request

- @RequestBody → Converts JSON into a Java object.
- @Valid → Checks the validation annotations.
- If validation fails → Spring automatically returns **400 Bad Request**.

Dependency required

If you are using Spring Boot, add this dependency to pom.xml:

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-validation</artifactId>  
</dependency>
```

real-world example

```
{  
    "name": "Laptop",  
    "price": 50000,  
    "stock": 10,  
    "category": "Electronics"  
}
```

Valid input → Product can be saved.

But:

```
{  
    "name": "",
```

```
"price": -100,  
"stock": -5,  
"category": ""  
}
```

Invalid input → Product should not be saved.

Validation protects your application from incorrect data before it reaches the service and database.

Step 7 — Create Global Exception Handler

A **Global Exception Handler** is a common place where we handle errors that occur in our application.

Without a global handler, every controller may need its own try-catch blocks. That makes the code repetitive.

With a global handler, we can handle errors in **one class** and return a standard response to the client.

@RestControllerAdvice

```
public class GlobalExceptionHandler {  
}
```

@RestControllerAdvice tells Spring:

“Whenever an exception occurs in any REST controller, check this class to see how it should be handled.”

Example

Suppose the client requests:

GET /api/v1/products/100

But product 100 does not exist.

Instead of returning an unclear error, our application can return:

```
{  
  "status": 404,  
  "error": "PRODUCT_NOT_FOUND",  
  "message": "Product not found"  
}
```

Exceptions we need to handle

Validation Exception

This occurs when the client sends **invalid product data**.

Example:

```
{  
  "name": "",  
  "price": -100  
}
```

If our DTO contains:

```
@NotBlank  
private String name;  
  
@Positive  
private BigDecimal price;
```

Spring detects that the data is invalid.

We should return: 400 Bad Request

Example response:

```
{  
  "status": 400,  
  "error": "VALIDATION_ERROR",  
  "message": "Invalid product data"  
}
```

The client sent incorrect data.

Product Not Found

This occurs when the requested product does not exist.

Example: GET /api/v1/products/999

If product 999 is not in the database, we throw:

```
throw new ProductNotFoundException("Product not found");
```

The global handler converts it into: 404 Not Found

Meaning: The requested product does not exist.

Duplicate Product

This occurs when the client tries to create a product that already exists.

Example: POST /api/v1/products

```
{  
  "name": "Laptop",  
  "price": 50000  
}
```

If "Laptop" already exists, we throw:

```
throw new DuplicateProductException("Product already exists");
```

The handler returns: 409 Conflict

The request conflicts with existing data.

Generic Exception

This handles unexpected errors that we have not specifically handled.

Example:

- Database connection failure
- Unexpected programming error
- Unknown server error

The handler returns: 500 Internal Server Error

Something went wrong on the server.

Step 8 — Create Standard Error Response

Instead of returning different error formats from different controllers, we return **one common format**.

Every error can contain:

```
{  
  "timestamp": "2026-09-08T10:00:00",  
  "status": 400,  
  "error": "VALIDATION_ERROR",  
  "message": "Invalid product data",  
}
```

```
"path": "/api/v1/products"
}
```

Explanation of each field

Field	Meaning
timestamp	When the error occurred
status	HTTP status code
error	Type of error
message	Human-readable explanation
path	API URL where the error occurred

The frontend or another microservice can easily understand the response.

Example:

```
if (response.status == 400) {
  // Show validation message
}
```

The response format remains consistent for all APIs.

Step 9 — Test All Error Scenarios

Now, we must **test each type of error and confirm that the API returns the correct HTTP status code and error message**.

Test 400 Bad Request

The client sent invalid data.

Example: Creating a product without a required name.

Request:

POST http://localhost:8083/products

Content-Type: application/json

```
{
  "name": "",
  "price": -100
}
```

Expected response:

400 Bad Request

```
{  
  "status": 400,  
  "message": "Validation failed",  
  "errors": {  
    "name": "Name is required",  
    "price": "Price must be greater than 0"  
  }  
}
```

Whether @Valid and your global exception handler correctly handle validation errors.

Test 404 Not Found

The requested product does not exist.

Example:

GET http://localhost:8083/products/999

If product 999 does not exist:

Expected response:

404 Not Found

```
{  
  "status": 404,  
  "message": "Product not found"  
}
```

Whether your service throws a ProductNotFoundException and the global handler converts it into a 404 response.

Test 409 Conflict

The request conflicts with existing data.

Example: Trying to create a product with a name that must be unique.

Request: POST http://localhost:8083/products

Content-Type: application/json

```
{  
  "name": "Laptop",  
  "price": 50000  
}
```

Expected response:

409 Conflict

```
{  
  "status": 409,  
  "message": "Product already exists"  
}
```

Whether duplicate-data errors are handled correctly.

Test 500 Internal Server Error

An unexpected error occurred inside the application.

Example: A database failure or an unexpected exception.

Expected response:

500 Internal Server Error

```
{  
  "status": 500,  
  "message": "Something went wrong"  
}
```

Whether your global exception handler catches unexpected exceptions and returns a standard error response instead of exposing internal details.

You can perform them in **Postman**.

Test	Method	URL	Expected Status
Invalid product data	POST	/products	400
Product not found	GET	/products/999	404
Duplicate product	POST	/products	409
Unexpected server error	Any applicable request	Your API	500

For the 500 test, you should not intentionally break your application in production. During development, you can simulate an unexpected exception if needed.